



《Mantis Standard人形机器人开发手册》

一、文档概述

编写目的

本文档用于**Mantis Standard人形机器人** 的：

- 客户使用说明与安全规范
- ROS 二次开发的入门级技术指导

通过本文件，交付方与客户可对机器人当前功能状态、性能边界、使用方式及开发接口形成一致认知，降低交付风险，提升后续使用与维护效率。

适用对象

- 客户研发工程师（ROS 初级~中级）
- 机器人系统集成与售后人员

机器人系统简介

本机器人为 **基于 ROS 2 的人形基础机器人系统**，整体由以下模块构成：

- 人形上肢机械臂系统（多自由度）
- 头部视觉系统
- 可移动底盘系统
- 锂电池供电系统

- 工业级电机与驱动系统
- 遥控操作系统（摇操作）
- ROS 2 控制与通信系统

机器人支持： - 遥控操作模式 - ROS 自动控制模式 - **二次开发扩展**

二、安全注意事项（重要）

1. 通用安全准则 (General Safety Principles)

- **授权操作**：本机器人仅限接受过基本 ROS 操作培训及阅读过本手册的人员操作。严禁儿童或未经培训的人员擅自触碰。
- **禁止私自改装**：严禁在未经原厂技术授权的情况下，拆卸机器人核心控制器、改变电路逻辑或修改电机的 PID 核心参数。私自改装造成的硬件烧毁及安全事故，厂家概不负责。
- **预热要求**：在低温环境下，建议先空载运行 3-5 分钟，待减速机润滑脂及电机进入工作温度后再加负载。

2. 机械风险与人体保护 (Mechanical Hazards)

- **【警告】夹伤与挤压**：机器人多关节联动具有极强的动力。严禁在机械臂运动范围内放置身体任何部位。特别注意**夹爪 (End-effector)** 和**肘部旋转关节**，其产生的剪切力足以造成骨折或严重挫伤。
- **【警告】惯性冲击**：人形机器人在搬运重物时，急停或快速转向会产生巨大的惯性扭矩。严禁在机器人高速行驶路径的正前方站人。
- **【提示】着装规范**：操作人员在维护机器人时，严禁佩戴领带、首饰或穿着过于宽松的衣物，防止被旋转关节或底盘驱动轮卷入。

3. 电气系统与电池维护 (Electrical & Battery Safety)

- **【核心禁令】严禁热插拔**：所有总线舵机接口、激光雷达接口、主控板电源线，必须在**关机断电**状态下进行插拔。带电操作产生的瞬时浪涌电流（Spike Current）是电子元件损坏的第一大原因。
- **【警告】电池物理防护**：动力电池位于机器人躯干中心。严禁使用尖锐物体刺入壳体。如发现机器人跌落后电池仓有变形、发烫、异味或冒烟现象，请立即将其移至开阔地带，切勿靠近。
- **【提示】充电规范**：
 - a. 必须使用随车配备的专用平衡充电器。
 - b. 建议充电环境温度保持在室温。
 - c. 严禁在环境湿度超过 80% 的状态下充电，以防接口短路。

4. 运行环境与地质限制 (Environmental Restrictions)

- **【警告】坡度限制：**由于人形机器人重心较高，本产品仅设计用于坡度小于 **10°** 的室内环境。在斜坡上强制执行抓取或举升动作极易导致失稳翻滚。
- **【警告】液体避让：**本机器人非防水设计。地面积水、油污、强酸碱液体均会损坏底盘驱动轮并可能造成内部电路短路。严禁在潮湿地面进行搬运作业。
- **【提示】电磁干扰：**严禁在大型配电箱、高功率微波发射塔或大型工业超声波设备旁运行。强干扰会导致 IMU 姿态解算偏差，造成机器人莫名摔倒。

5. ROS 软件安全与应急处置 (Software & Emergency Response)

- **【强制】急停机制：**操作员必须熟练掌握遥控器上的“急停 (E-Stop)”键位置。在程序逻辑死循环、网络心跳丢失或机器人动作失控时，应第一时间物理切断动力电源。
- **【警告】网络安全风险：**若机器人连接公网，请务必修改默认 SSH 密码。严禁在不稳定的公共 WiFi 环境下控制机器人移动，防止因指令丢包造成的运动失控。
- **【提示】日志监控：**在执行高风险测试（如极限负载）时，建议通过 `ros2 node list` 和 `ros2 topic echo /对应话题` 实时监控各电机扭矩和温度报警状态。

6. 长期存放与维护要求 (Storage & Maintenance)

- **长期存放：**如机器人停用超过 30 天，请将电池取下或断开电源物理总闸。每月需进行一次维护性充电，电量保持在 **60%** 左右。
- **紧固检查：**人形机器人高频往复运动会导致关节螺丝松动。建议每工作 **50 小时** 使用螺纹胶对关键部位的 M3/M4 螺丝进行二次紧固。

三、代码仓库说明

1. 二次开发SDK

https://github.com/BlueWorm-EAI-Tech/bw_std_SDK
github.com

2. OTA镜像仓库



 https://github.com/BlueWorm-EAI-Tech/bw_system

GitHub - BlueWorm-EAI-Tech/bw_system: 用于ota的仓库
用于ota的仓库. Contribute to BlueWorm-EAI-Tech/bw_system development by creating an account on GitHub.

3. motion动作规划

https://github.com/BlueWorm-EAI-Tech/bw_std_motion
github.com

4. 二次开发ROS2

https://github.com/BlueWorm-EAI-Tech/bw_std_teleop_ros2
github.com

5. urdf结构模型

https://github.com/BlueWorm-EAI-Tech/bw_std_description.git
github.com

仓库说明：

1. 二次开发SDK

- a. 功能：基于 Zenoh 的 Mantis 机器人控制 SDK，无需安装 ROS2，即可完成相关功能开发；

2. OTA镜像仓库

- a. 功能：OTA更新Docker镜像，提供后期维护升级；

3. motion动作规划

- a. 功能：实现机器人动作操作组，进行录制回放；

4. ROS2仓库

- a. 功能：实现机器人遥操桥接，运动学解析，ros2_control控制，机器人遥操作ROS2链路

5. urdf结构模型

- a. 功能：多种工程形态，可用于多场景开发，包括VLA，SLAM等

四、Linux通用工具链

终端工具

开发默认使用zsh作为shell

代码块

```
1 sudo apt update
2 sudo apt install zsh
```

```
3  chsh -s $(which zsh)
4  exec zsh
```

下载常用开发工具

代码块

```
1  sudo apt install -y \
2      vim build-essential cmake ninja-build gdb clang clangd ccache pkg-config \
3      git git-lfs curl wget unzip zip tar ripgrep tmux htop jq tree ca-
    certificates \
4      python3 python3-pip python3-venv pipx locales openssh-server
```

启动sshd以方便远程连接

代码块

```
1  sudo sshd -t
2  sudo systemctl start ssh
3  sudo systemctl status ssh --no-pager
4  sudo systemctl enable ssh
```

配置Docker镜像加速

代码块

```
1  sudo vim /etc/docker/daemon.json
```

粘贴如下内容并使用

代码块

```
1  {
2      "dns": ["223.5.5.5", "1.1.1.1"],
3      "registry-mirrors": [
4          "https://registry-1.docker.io",
5          "https://mirror.ccs.tencentyun.com",
6          "https://docker.1ms.run",
7          "https://docker.1panel.live",
8          "https://docker.m.ixdev.cn",
9          "https://hub.rat.dev",
10         "https://docker.xuanyuan.me",
11         "https://docker.xuanyuan.run",
12         "https://docker.xuanyuan.dev",
```

```
13     "https://dockerproxy.net",
14     "https://docker-registry.nmqu.com",
15     "https://docker.hlmirror.com",
16     "https://hub3.nat.tf",
17     "https://hub4.nat.tf",
18     "https://docker.m.daocloud.io",
19     "https://docker.kejilion.pro",
20     "https://hub.lpanel.dev",
21     "https://dockerproxy.cool",
22     "https://docker.apiba.cn",
23     "https://proxy.vvvv.ee",
24     "https://docker.fnnas.com"
25 ]
26 }
```

重启docker以生效

代码块

```
1 sudo systemctl restart docker
```

Docker常用指令

- 镜像相关

代码块

```
1  #搜索镜像
2  docker search 镜像名称
3  #查看镜像
4  docker images
5  #查看容器
6  docker ps
7  #运行镜像
8  docker run 镜像名/镜像ID
9  #拉取镜像
10 docker pull 镜像名:tag
11 #删除镜像
12 docker rmi -f 镜像名/镜像ID
```

- 容器相关

代码块

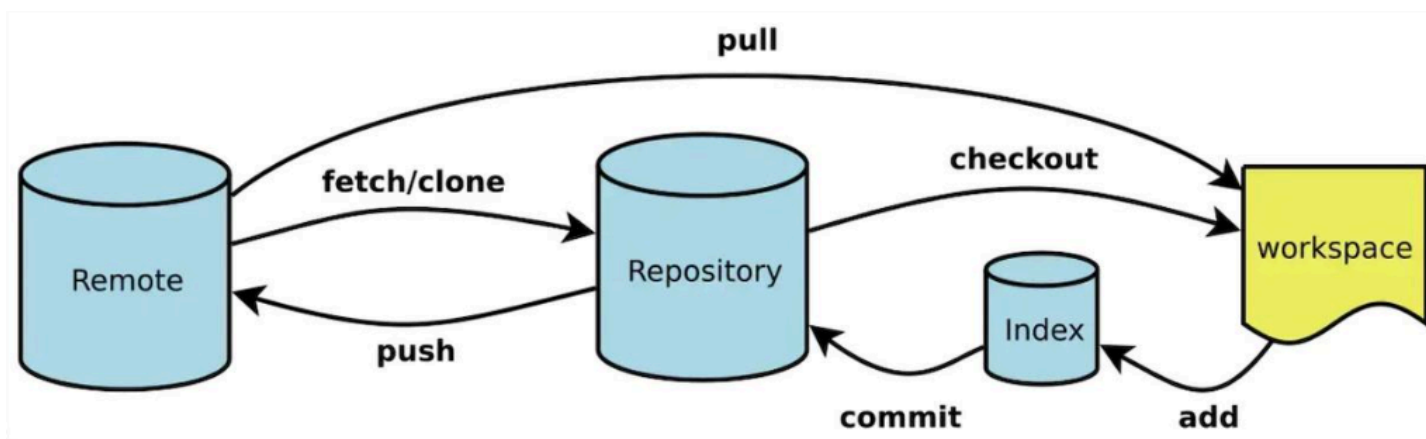
```
1  #关闭容器
```

```

2  docker stop 容器名/容器ID
3  #启动容器
4  docker start 容器名/容器ID
5  #删除容器
6  docker rm -f 容器名/容器ID
7  #进入容器内部
8  docker exec -it 容器名/容器ID bash
9  #查看容器信息
10 docker inspect 容器名/容器ID
11 #查看容器日志
12 docker container logs 容器名/容器ID
13 #从容器内复制文件
14 docker cp 容器名/容器ID:docker容器内的文件地址 docker所在宿主机的文件地址

```

Git通用教程



1)Git基本概念

- **Workspace**：工作区
- **Index/Stage**：暂存区
- **Repository**：本地仓库
- **Remote**：远程仓库

2)创建Git版本库

- 创建Git空目录

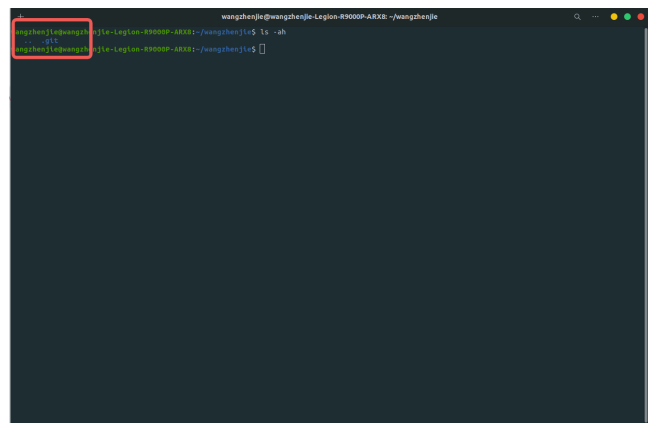
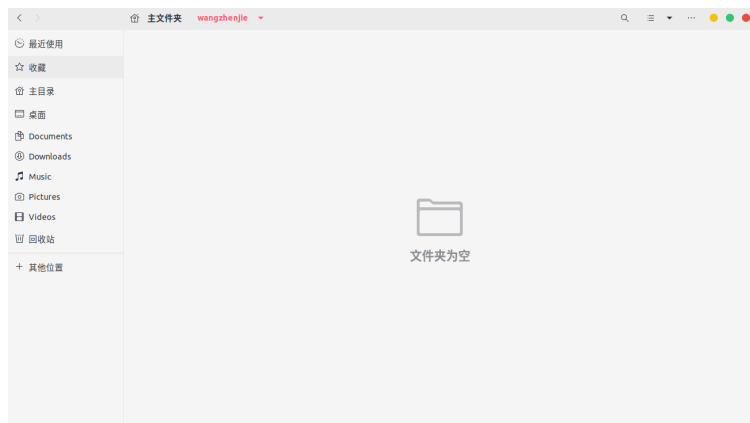
代码块

```

1  mkdir wangzhenjie #自己的Git工作区
2  cd wangzhenjie
3  #使用Git初始化命令
4  git init
5  #已初始化空的 Git 仓库于 /home/wangzhenjie/wangzhenjie/.git/

```

初始化成功之后，会生成一个.git目录，这个目录是Git用来管理版本库的；



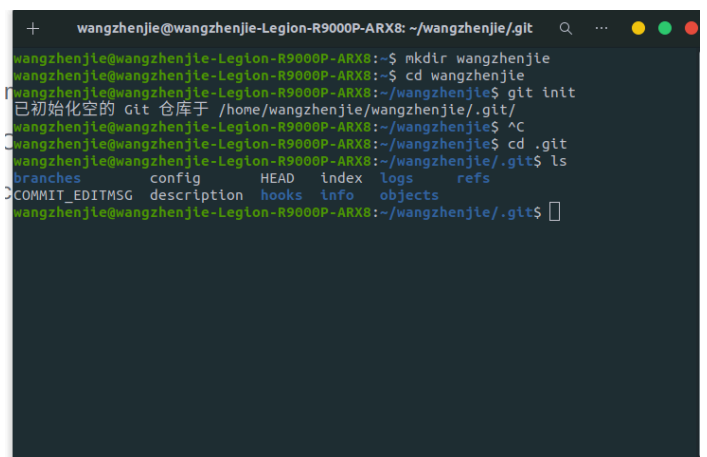
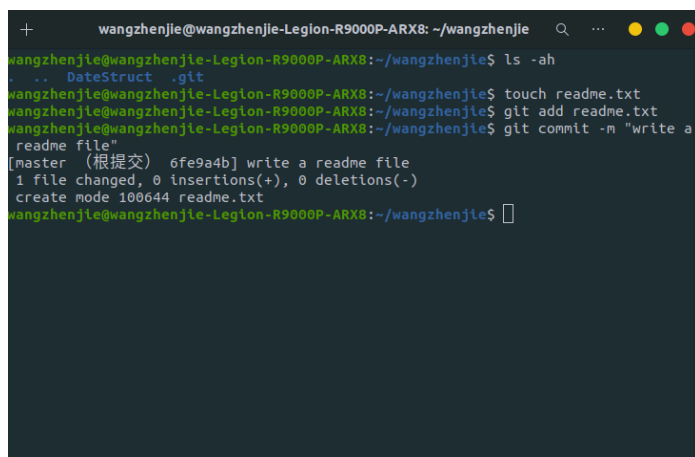
• 添加文件至版本库

版本控制系统只能追踪文件的改动，可以告诉你每次改了啥东西，但是不知道具体改了啥，因为一些特性，使用Git上传文本文件的时候最好使用UTF-8编码（不要使用Win自带的记事本打开文件文件）

1. 编写readme文件（提交流程）

代码块

```
1 touch readme.txt #存放在wangzhenjie目录下
2 #将readme文件添加到仓库（暂存区）中
3 git add readme.txt #将readme文件提交到仓库（此时是暂存区）
4 #添加完成之后是没有任何显示的
5
6 #提交到本地仓库
7 git commit -m "write a readme file" #执行成功之后会显示下面的内容
8 #[master (根提交) 6fe9a4b] write a readme file
9 #1 file changed, 0 insertions(+), 0 deletions(-)
10 #create mode 100644 readme.txt
```



2. add和commit的关系

工作目录

wangzhenjie (本地工作区域)

file1

add

file2

add

file3

add

file4

add

暂存区

git commit

本地仓库

通过git add将每一次单独修改的文件先存放到缓冲区，当多个文件全部修改完成之后，此时缓冲区就有了很多的被add的文件，然后最后一次性使用**git commit**提交到henjie-Legion-R9000P-ARX8:~/catkin_ws\$ git init本地仓库。



【使用场景】当一个项目有很多个文件需要修改的时候，当你修改完一个之后，就顺手把他add到缓冲区，当几个修改完之后缓冲区就会有全部的已经修改的文件，最后一次性使用commit提交，会方便很多。

提交补充

- Git命令只能在Git仓库下面执行（Git init除外），在仓库外是没有意义的；
- Git只能提交在Git仓库下的文件

3)Git基础使用

(1) Git常用修改指令

- git status** 显示仓库当前状态

当我们修改了文件内容之后,使用git status 查看当前状态，（此时显示我们有一个被修改过但是还没有提交的修改），虽然我们可以知道修改了，但是如果想知道修改了什么内容就需要使用到git diff来查看上一次修改前的地方；

```
git status
```

```
1  位于分支 master
2  尚未暂存以备提交的变更：
3      （使用 "git add <文件>..." 更新要提交的内容）
4      （使用 "git restore <文件>..." 丢弃工作区的改动）
5      修改：      readme.txt
```

6

7 修改尚未加入提交（使用 "git add" 和/或 "git commit -a"）

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git status
位于分支 master
尚未暂存以备提交的变更:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
    修改:   readme.txt

修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git status
位于分支 master
尚未暂存以备提交的变更:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
    修改:   readme.txt

修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git diff
diff --git a/readme.txt b/readme.txt
index 19dbb0b..c2c5987 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,1 @@
-Git is a version control
\ No newline at end of file
+Git is a version control wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

当我们git add之后，再使用git status查看状态的时候就会发现下图所示。

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git status
位于分支 master
尚未暂存以备提交的变更:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
    修改:   readme.txt

修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git diff
diff --git a/readme.txt b/readme.txt
index 19dbb0b..c2c5987 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,1 @@
-Git is a version control
\ No newline at end of file
+Git is a version control wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git add readme.txt
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git status
位于分支 master
要提交的变更:
  (使用 "git restore --staged <文件>..." 以取消暂存)
    修改:   readme.txt

wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git status
位于分支 master
尚未暂存以备提交的变更:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
    修改:   readme.txt

修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git diff
diff --git a/readme.txt b/readme.txt
index 19dbb0b..c2c5987 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,1 @@
-Git is a version control
\ No newline at end of file
+Git is a version control wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

• git diff 查看上一版修改内容

git diff

```
1 diff --git a/readme.txt b/readme.txt
2 index 19dbb0b..c2c5987 100644
3 --- a/readme.txt
4 +++ b/readme.txt
5 @@ -1,1 @@
6 -Git is a version control
7 \ No newline at end of file
8 +Git is a version control wangzhenjie
```

• git log 查看Git提交的日志

git log

```
1 Author: wangzhenjie <1345611597@qq.com>
2 Date: Wed Apr 23 14:23:14 2025 +0800
```

```

3
4     the 3 change
5
6     commit 8d29e7a9a6c210e3ccbc89905aa3db26ce0e8a64
7     Author: wangzhenjie <1345611597@qq.com>
8     Date:   Wed Apr 23 11:46:30 2025 +0800
9
10    update readme
11
12    commit 6fe9a4b56ab5ad0022eec3db65acbbf607b82bee
13    Author: wangzhenjie <1345611597@qq.com>
14    Date:   Wed Apr 23 11:17:19 2025 +0800
15
16    write a readme file

```

(2) Git回退版本

- git reset --hard **HEAD^**

其中[^]表示回退到上一个版本，两个^^表示回退到上上个版本，当回退的数字比较多时候采用**HEAD~【数字】**来进行表示。

代码块

```
1 git reset --hard HEAD^ #回到上一版本
```

--hard 指回退到上个版本已经提交的状态

--soft 指回退到上个版本暂未提交的状态

--mixed 指回退到上个版本已经添加至缓冲区但是未提交的状态

```

write a readme file
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git log
commit d2e10d5f208dae9f20b7c5fcaec945b1cc769552 (HEAD -> master)
Author: wangzhenjie <1345611597@qq.com>
Date:   Wed Apr 23 14:23:14 2025 +0800

    the 3 change

commit 8d29e7a9a6c210e3ccbc89905aa3db26ce0e8a64
Author: wangzhenjie <1345611597@qq.com>
Date:   Wed Apr 23 11:46:30 2025 +0800

    update readme

commit 6fe9a4b56ab5ad0022eec3db65acbbf607b82bee
Author: wangzhenjie <1345611597@qq.com>
Date:   Wed Apr 23 11:17:19 2025 +0800

    write a readme file
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git reset --hard HEAD^
HEAD 现在位于 8d29e7a update readme
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$

```

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git log
commit d2e10d5f208dae9f20b7c5fcaec945b1cc709552 (HEAD -> master)
Author: wangzhenjie <1345611597@qq.com>
Date: Wed Apr 23 14:23:14 2025 +0800

    the 3 change

commit 8d29e7a9a6c210e3ccbc89905aa3db26ce0e8a64
Author: wangzhenjie <1345611597@qq.com>
Date: Wed Apr 23 11:46:30 2025 +0800

    update readme

commit 6fe9a4b56ab5ad0022eec3db65acbbf607b82bee
Author: wangzhenjie <1345611597@qq.com>
Date: Wed Apr 23 11:17:19 2025 +0800

    write a readme file
```

回退前Log文件

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git log
commit 8d29e7a9a6c210e3ccbc89905aa3db26ce0e8a64 (HEAD -> master)
Author: wangzhenjie <1345611597@qq.com>
Date: Wed Apr 23 11:46:30 2025 +0800

    update readme

commit 6fe9a4b56ab5ad0022eec3db65acbbf607b82bee
Author: wangzhenjie <1345611597@qq.com>
Date: Wed Apr 23 11:17:19 2025 +0800
```

回退后Log文件

- git reset --hard + 【版本号】

其中版本号表示我们想回到的版本，也就是说当回退之后还是可以回到之前的新版本的；

- git relog 查询版本号

代码块

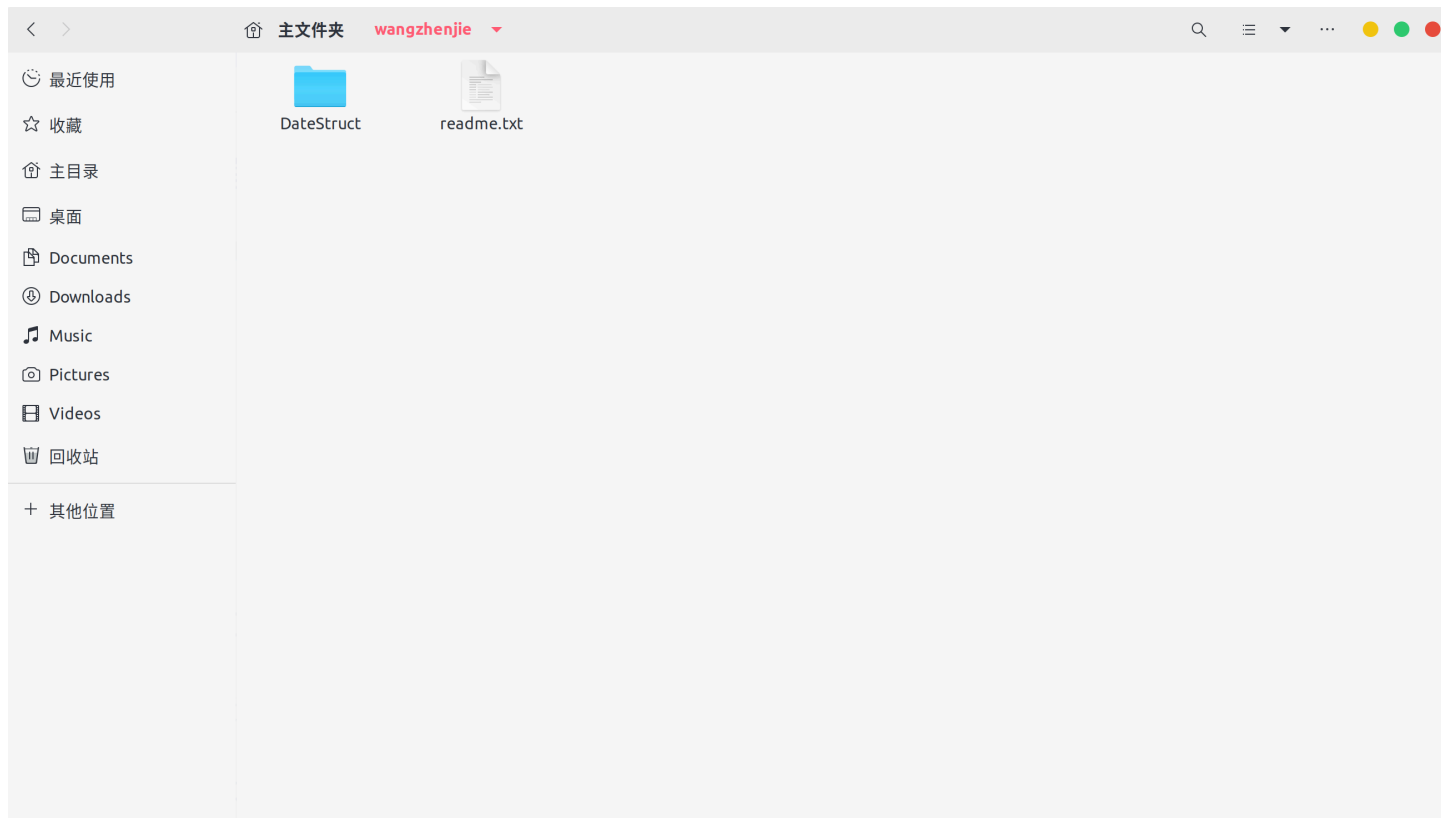
```
1  git relog #查看每一次的版本号
2  8d29e7a (HEAD -> master) HEAD@{0}: reset: moving to HEAD^
3  d2e10d5 HEAD@{1}: commit: the 3 change
4  8d29e7a (HEAD -> master) HEAD@{2}: commit: update readme
5  6fe9a4b HEAD@{3}: commit (initial): write a readme file
```

```
+ wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie  🔍 ... ● ● ●

wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git relog
git: 'relog' 不是一个 git 命令。参见 'git --help'。

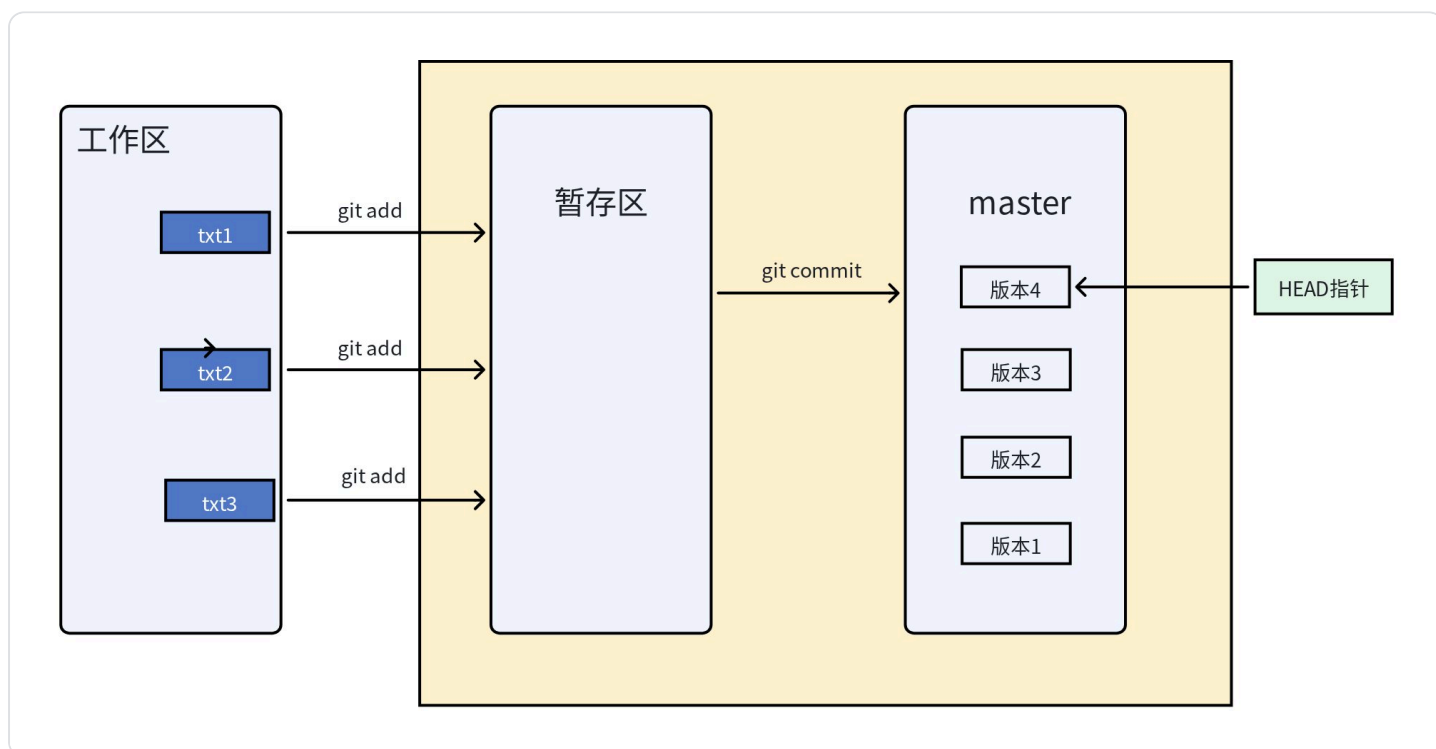
最相似的命令是
  relog
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git relog
8d29e7a (HEAD -> master) HEAD@{0}: reset: moving to HEAD^
d2e10d5 HEAD@{1}: commit: the 3 change
8d29e7a (HEAD -> master) HEAD@{2}: commit: update readme
6fe9a4b HEAD@{3}: commit (initial): write a readme file
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

(3) 工作区和暂存区



- **工作区**：在电脑中可以看到的目录
- **版本库（暂存区index）**：

版本库就是工作区的一个隐藏文件夹（.git文件）他包括了我们的暂存区，还有我们的master分支，还有指向master的指正HEAD；



- step1 使用git add将文件添加到暂存区中；
- step2 git commit 将暂存区的文件一次性提交到当前分支（也就是本地仓库）；
- 版本回退实际上就是改变HEAD指正的指向；

代码块

```
1 git diff HEAD -- readme.txt #查看两种
```

(4) 修改管理

- 撤销修改

- git checkout --file 丢弃对工作区的更改

代码块

```
1 git checkout --file #--很重要，如果没有这个就会变成切换分支的命令；
```

分为两种情况：

1. readme.txt 修改后还没有被git add到暂存区，使用checkout命令后会回到和版本库一模一样的状态；
2. readme.txt已经被添加到了暂存区，**之后又进行了修改**，现在使用checkout命令会返回到添加到暂存区后的状态；

git checkout这个命令就是让文件回到最近一次git add或git commit之前的状态；

【特殊情况】

git reset HEAD <file>可以撤销掉暂存区的设置，重新回到工作区；

当你已经添加到暂存区之后（再没有进行修改），想进行回退，解决办法——>**先使用git reset** 回退到修改之前的版本，**再使用git checkout 撤销修改**；

如果提交到了版本库（本地仓库），直接使用git reset 回退到修改之前的版本；

 但是，如果你已经远程提交了，那就BBQ了

- 删除文件

在Git中删除文件也是一个**修改操作**；

代码块

```
1 #在本地删除test.c
2 rm test.c #删除test.c
3 #查询版本库当前状态
4 git status
```

会提示有文件删除

git日志如下

- 1 位于分支 master
- 2 尚未暂存以备提交的变更：
- 3 (使用 "git add/rm <文件>..." 更新要提交的内容)
- 4 (使用 "git restore <文件>..." 丢弃工作区的改动)
- 5 删除: test.c
- 6
- 7 未跟踪的文件：
- 8 (使用 "git add <文件>..." 以包含要提交的内容)
- 9 DateStruct/
- 10 c/
- 11
- 12 修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")

使用git rm 更新操作

代码块

- 1 git rm test.c #将目标文件从暂存区删除
- 2 git commit #提交删除操作

```
commit 7109c9742d578834ba47ec2acb69e4b0dc496b4b (HEAD -> master)
```

```
Author: wangzhenjie <1345611597@qq.com>
```

```
Date: Fri May 2 18:03:29 2025 +0800
```

```
remove test.c
```

```
commit 946645b8fd11427491b2c5bb014be1426989ba7c
```

```
Author: wangzhenjie <1345611597@qq.com>
```

```
Date: Fri May 2 17:59:18 2025 +0800
```

```
test.c
```

```
commit 8d29e7a9a6c210e3ccbc89905aa3db26ce0e8a64
```

```
Author: wangzhenjie <1345611597@qq.com>
```

```
Date: Wed Apr 23 11:46:30 2025 +0800
```

```
update readme
```

```
commit 6fe9a4b56ab5ad0022eec3db65acbbf607b82bee
```

```
Author: wangzhenjie <1345611597@qq.com>
```

```
Date: Wed Apr 23 11:17:19 2025 +0800
```

```
write a readme file
```

```
:|
```

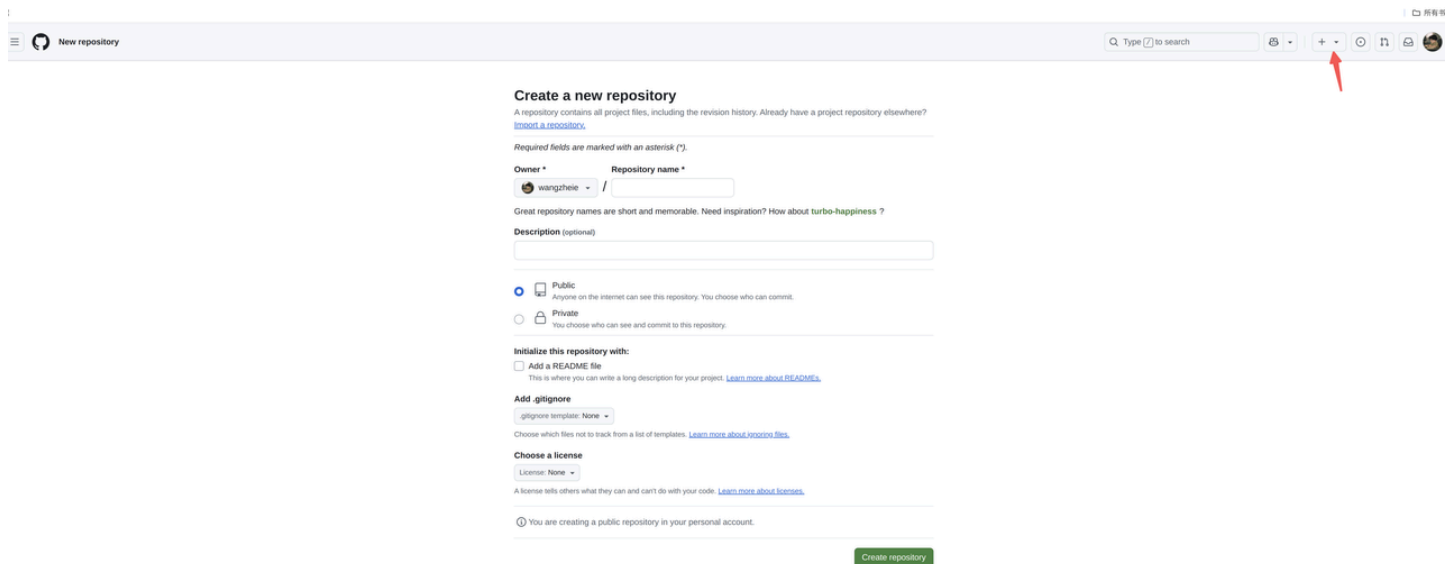
4)Git远程仓库

怎么配置自己的Git仓库在这个文档里（添加秘钥）

- 使用远程库



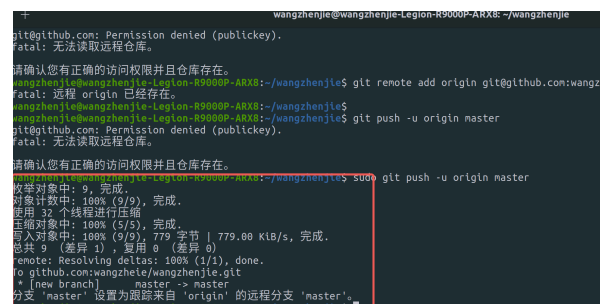
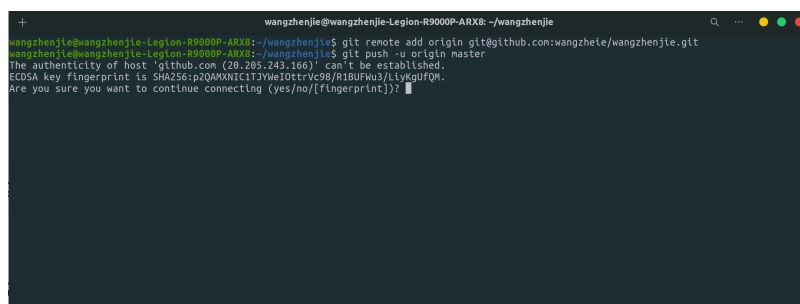
一般来说，本地有一个Git仓库，又在GitHub创建了一个Git仓库，并且两个仓库远程同步，这样GitHub既可以作为备份，也可以让其他人通过该仓库来进行协作开发；



代码块

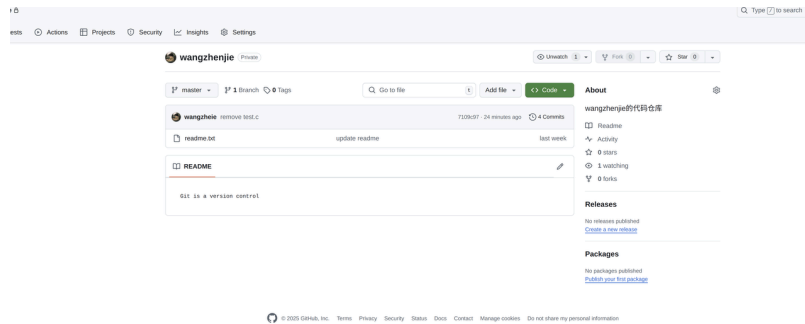
```
1 git@github.com:wangzheie/wangzhenjie.git #我的Git地址
2
3 git remote add origin git@github.com:wangzheie/wangzhenjie.git
4
5 sudo git push -u origin master#第一次使用，将默认的全部库都传上去
6
7 sudo git push origin master #之后上传仓库使用这个即可
```

由于远程库是空的，我们第一次推送 `master` 分支时，加上了 `-u` 参数，Git不但会把本地的 `master` 分支内容推送的远程新的 `master` 分支，还会把本地的 `master` 分支和远程的 `master` 分支关联起来，在以后的推送或者拉取时就可以简化命令。



```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8: ~/wangzhenjie
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ sudo git push -u origin master
分支'master' 设置为跟踪来自 'origin' 的远程分支 'master'。
Everything up-to-date
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git push origin master
git@github.com: Permission denied (publickey).
fatal: 无法读取远程仓库。

请确认您有正确的访问权限并且仓库存在。
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ sudo git push origin master
Everything up-to-date
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```



• 删除远程库

代码块

```
1  #step1 查看远程库信息
2  git remote -v
3  origin          git@github.com:wangzheie/wangzhenjie.git (fetch)
4  origin          git@github.com:wangzheie/wangzhenjie.git (push)
5  #step2 删除远程库
6  git remote rm origin
7
```

• 从远程库克隆

知道仓库地址后直接git clone

代码块

```
1  git clone +网址 #最经典的一个
```

5)分支管理

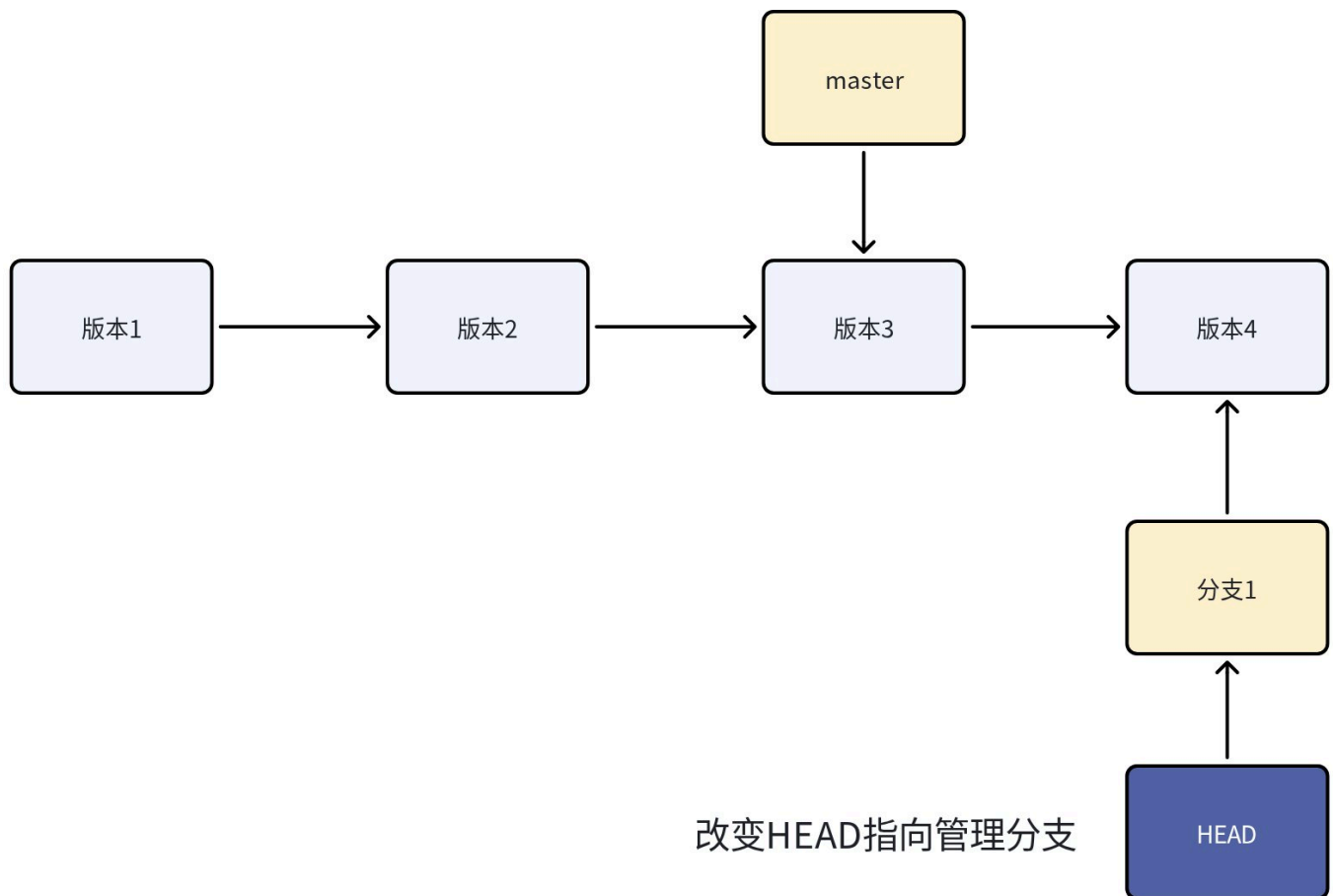


分支管理：创建分支之后，直到开发完成之后，一次性合并到原来的分支上，不会影响其余人工作；

相比于SVN管理，Git在1秒钟之内就可以完成，无论版本库是一个文件还是一万个文件，Git拉起分支的速度特别快；

(1) 创建与合并分支

分支的创立实际上就是改变指针的指向，这样只需要该表指针指向的分支就可以快速切换不同的分支了；



- 新建分支&&切换分支

代码块

```
1  #新建分支
2  git branch dev
3  #git branch 查看当前分支 当前分支会带一个*号
4
5  #切换分支
6  git checkout dev
7  #合并版本
8  git checkout -b dev
```

- 分支合并git merge

代码块

```
1  #git merge 用于合并指定分支到当前分支
2  git merge dev #合并dev分支到master
3  #合并后删除新建分支
4  git branch -d dev #保留一个分支
```

- **git switch**

代码块

```
1  #创建并切换到新的分支
2  git switch -c dev #切换创建新分支
3  #切换分支
4  git switch master
```

(2) 分支冲突

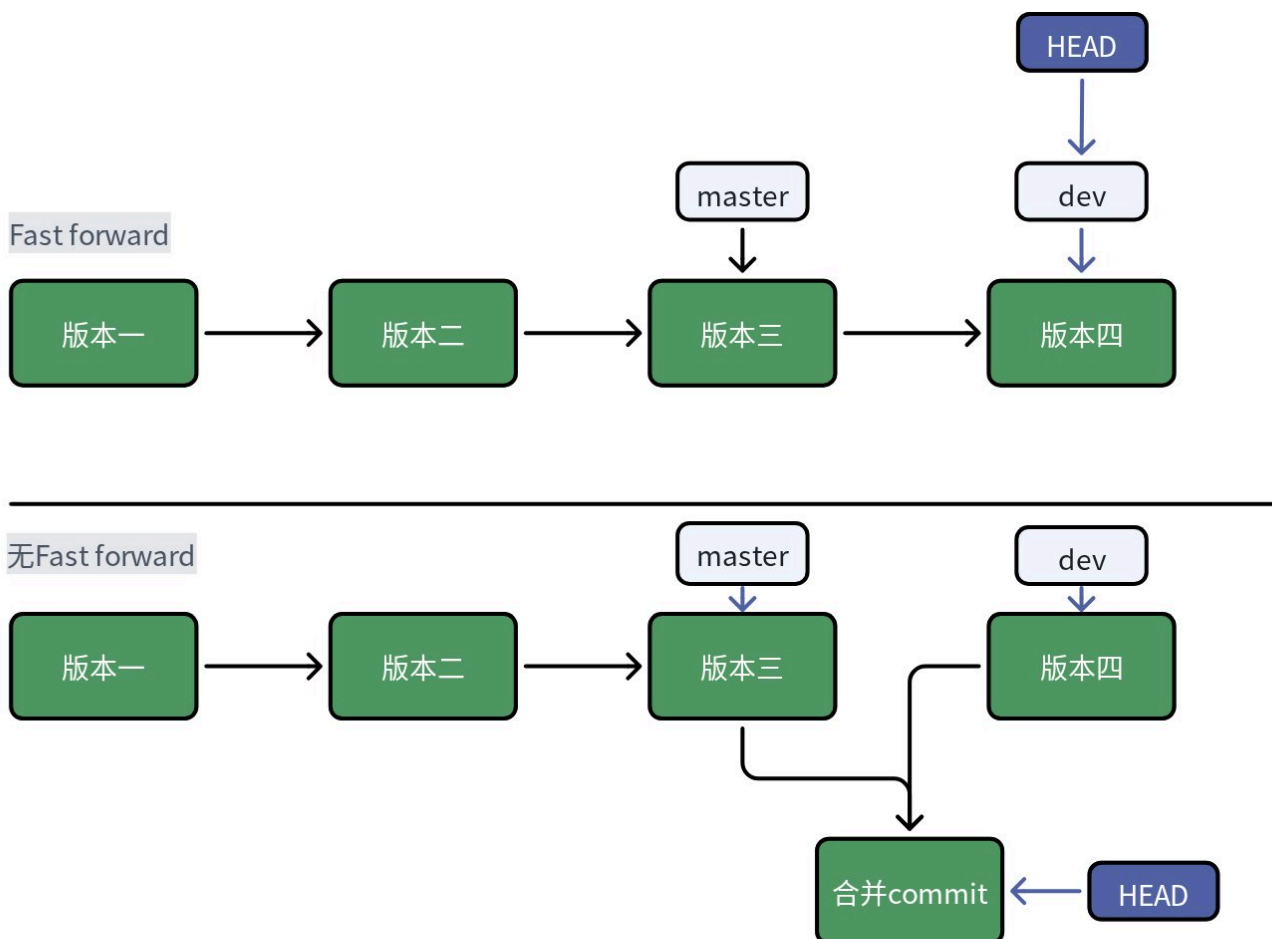
- **Fast-forward快速模式**

Git默认使用Fast-forward快速模式，指直接把master指向dev当前的提交，使用这种模式，删除掉分支后会直丢掉分支信息；

- `--no--ff` 模式禁用Fast-forward模式,处于禁用模式的时候，使用该模式的时候各个分支会合并一个新的commit进行提交，这样就保留了我们的分支记录；

代码块

```
1  git merge --no--ff -m "merge with no --ff "
```



- **Bug修复模式（实践一下）**

Git较为强大的分支功能，在我们需要修复Bug的时候，新建一个临时分支来修复，合并后删除临时分支即可

- Git stash 储藏命令

代码块

```
1  #临时存储当前工作环境
2  git stash #临时存储
3  Saved working directory and index state WIP on dev: f52c633 add merge
4  #列出临时工作区保留
5  git stash list
6  stash@{0}: WIP on dev: f52c633 add merge #可以根据stash@{0}选择保存的哪一个工作区
7  #方法1 恢复工作区初始状态
8  git stash apply #此时stash的内容不会删除需要使用git stash drop删除
9  git stash drop #删除保存的工作区
10 #方法2 恢复工作区初始状态
11 git stash pop #恢复的同时把stash内容也删除了
```

- 重复Bug修复 cherry-pick命令（不同分支转移）



对于两个不同的代码分支来说：

如果需要全部变动则使用git merge，但是只是需要部分的代码改动只需要使用git cherry pick

代码块

```
1  cherry-pick commitID号 #合并某一个特定分支
```

执行完cherry pick之后，会产生一个新的提交，新提交的哈希值和原来的不一样，但是标识名一样；只是改动一样，是两个完全不同的commit；

6)Git标签管理



Git本来的commit是一系列奇怪的数字，在查找的时候不太方便，所以就创建的标签，通过起一些有意义的名字，把他和指定的commit绑定在一起；

(1) 创建标签

- 切换到需要打标签的分支

代码块

```
1 git branch #切换分支
2 Switched to branch 'master' #切换到目标分支
```

- 使用Git tag命令打标签

代码块

```
1 #查看标签
2 git tag #列出全部标签
3 #打标签
4 git tag <name> #默认打到了最新的commit上
5 git tag <name> commit_ID #给之前的commit打版本号
```

标签不是按照时间顺序列出，而是按照字母排序，可以使用git show查看标签信息；

- 使用Git tag命令打标签（特殊版）

代码块

```
1 git tag -a <name> -m "说明文字" #创建带有说明的标签
2 -a 指定标签名 -m 指定说明文字
```

```
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$ git show V1.0
commit 946645b8fd11427491b2c5bb014be1426989ba7c (tag: V1.0)
Author: wangzhenjie <1345611597@qq.com>
Date: Fri May 2 17:59:18 2025 +0800

    test.c

diff --git a/test.c b/test.c
new file mode 100644
index 0000000..e69de29
wangzhenjie@wangzhenjie-Legion-R9000P-ARX8:~/wangzhenjie$
```

(2) 操作标签

- 删除标签（本地）

代码块

```
1 git tag -d <tagname>
2 #推送标签到远程
3 git push 删除标签（本地） <tagname>
4 git push origin --tags #一次性推送全部本地标签
```

- 删除标签（远程）

代码块

```
1  #Step1 本地删除
2  git tag -d <tagname>
3  #Step2 远程删除
4  git push 删除标签（本地） :refs/tags/<tagname>
```

五、二次开发教程

ROS2基础知识（参考培训教材[目00_课程大纲](#)）

- [目00_课程大纲](#)
 - [目01_ROS2安装与环境配置](#)
 - [目02_ROS2包的创建](#)
 - [目03_ROS2包的构建](#)
 - [目04_RViz可视化](#)
 - [目05_URDF模型](#)
 - [目06_TF坐标变换](#)
 - [目07_IK逆运动学](#)
 - [目08_VR遥操作介绍](#)
 - [目09_Mantis_Sim2Real实战](#)

OTA在线升级教程

1)OTA升级

- 目录结构说明：开发与部署 90% 的工作都在 `.env` 文件中完成；

代码块

```
1  /home/lanchong/bw_ws/bw_system
2  └── docker-compose.yml      # 容器编排文件
3  └── .env                    # 核心配置文件（必须修改）
4  └── README.md              # 官方说明
5  └── scripts/               # 启动/辅助脚本
```

- 系统概述

BW System 是一套基于 **Docker + WebRTC + 机器人控制中间件** 的模块化系统，支持：

-  VR 遥操作
-  SDK 指令桥接
-  Action / Motion 远程控制
-  OTA版本更新

系统通过 **统一的 Docker 镜像** + `.env` **配置文件** 管理不同运行模式，做到：一套镜像，多种模式，一键切换。

2)配置手册

1 镜像版本配置

代码块

```
1  VERSION=26.1.7.1
```

- 决定当前系统使用的 Docker 镜像版本
- 升级系统时只需修改此字段

2 网络配置（必须正确）

代码块

```
1  STATIC_IP=192.168.50.151
2  GATEWAY=192.168.50.1
3  DNS_SERVERS=8.8.8.8,1.1.1.1
4  HOST_IP=192.168.50.151
```

说明：

- `STATIC_IP`：机器人本机静态 IP（CIDR 格式）
- `GATEWAY`：局域网网关
- `DNS_SERVERS`：域名解析服务器
- `HOST_IP`：WebRTC / VR 通信使用的 IP（必须与 `STATIC_IP` 对应）

 **IP 错误会导致：VR 画面黑屏 / 连接失败 / SDK 无法通信**

3 WiFi 配置

代码块

```
1  PREFERRED_WIFI=lanchong-Mantis-5G
2  FALLBACK_WIFI=lanchong-5G
3  SUDO_PASSWORD=bwet
```

作用：

- 系统启动时自动连接首选 WiFi
- 首选失败时自动切换备用 WiFi
- `SUDO_PASSWORD` 仅用于网络配置（不会上传）

4 运行模式（核心）

代码块

```
1  RUN_MODE=real
```

运行模式决定 容器启动后自动运行哪些节点。

模式名称	值	说明
VR 遥操作	<code>real / vr</code>	默认模式，完整 VR 遥操作
SDK 桥接	<code>sdk / sdk_bridge</code>	接收 SDK 指令控制实机
动作远程	<code>motion_remote / action_module</code>	Action Module 服务代理
空闲模式	<code>idle / manual</code>	不自动启动节点（开发调试用）

💡 开发阶段强烈推荐使用 `idle` 模式。

5 WebRTC 摄像头配置

✅ 推荐方式：设备节点绑定

代码块

```
1  LEFT_DEVICE=/dev/video0
2  RIGHT_DEVICE=/dev/video2
```

- 稳定
- 不受 USB 枚举顺序影响

查看设备：

代码块

```
1  ls /dev/video*
```

⚠ 备用方式：USB Bus ID

代码块

```
1  LEFT_USB=usb-0000:00:14.0-4
2  RIGHT_USB=usb-0000:00:14.0-3
```

仅在 设备节点不稳定时使用。

3)系统启动流程

1 默认行为

- 系统 开机自动启动 Docker
- 默认进入 VR 遥操作模式

2 修改运行模式

代码块

```
1  vim .env
2  # 修改 RUN_MODE
3  RUN_MODE=idle
```

保存后 重启机器人或重启容器 生效。

3 手动启动系统

代码块

```
1  cd /home/lanchong/bw_ws/bw_system
2  docker compose up -d
```

4)系统升级教程

查看是否有新版本

- 进入 GitHub 官方仓库
 - 查看最新 `VERSION`
-

修改版本号

代码块

```
1 VERSION=26.1.8.1
```

拉取最新镜像

代码块

```
1 docker compose pull
```

重启系统

代码块

```
1 docker compose down
2 docker compose up -d
```

5)镜像与容器管理

拉取镜像

代码块

```
1 docker compose pull
```

更新并重启

代码块

```
1  docker compose pull
2  docker compose down
3  docker compose up -d
```

指定版本回滚

代码块

```
1  VERSION=26.1.7.0
2  docker compose pull
3  docker compose up -d
```

Mantis SDK 开发教程

本教程面向使用 **Mantis 机器人控制 SDK**（基于 Zenoh，无需 ROS2）的开发者，旨在帮助你快速掌握 **机器人控制、API 调用、并行运动以及反馈处理**。

代码块

```
1  mantis

2      |—__init__.py
3      |—_validation.py

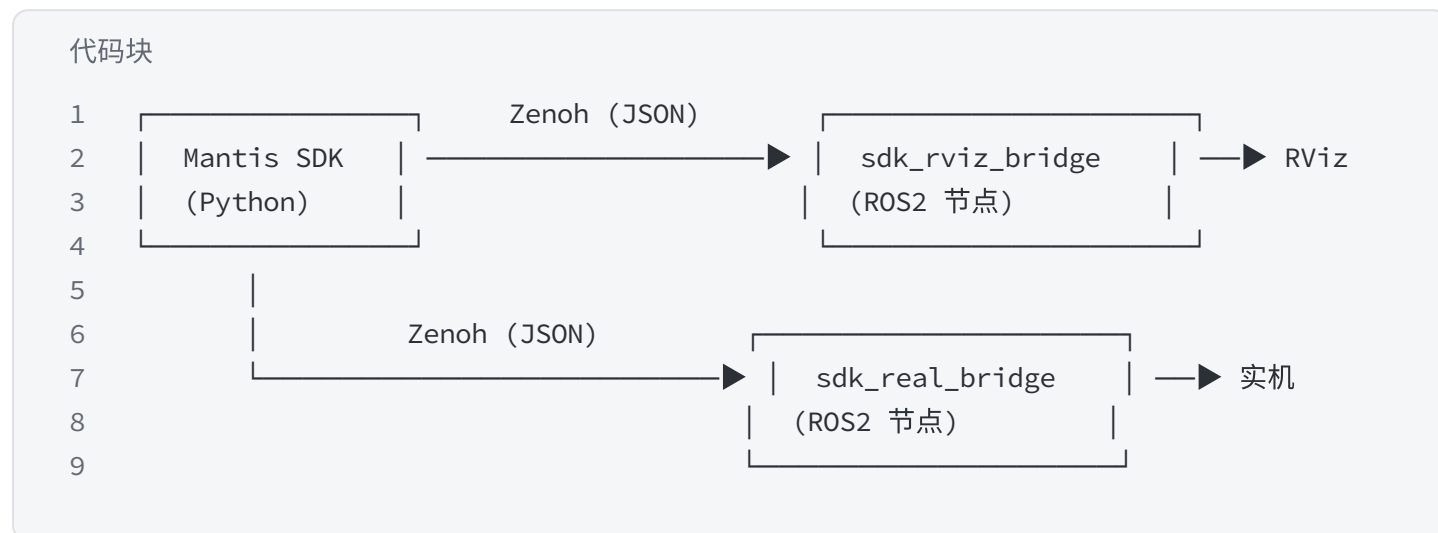
4      |—arm.py
5      |—chassis.py
6      |—constants.py
7      |—discovery.py
8      |—gripper.py
9      |—head.py
10     |—mantis.py
11     |—waist.py
```

1) 系统概述

Mantis SDK 是一个纯 **Python + Zenoh** 的控制库，特点：

- 🚀 无需安装 ROS2，直接 Python 调用
- 🤖 完整控制：双臂、夹爪、头部、腰部、底盘
- 🎯 同一套接口驱动 RViz 仿真和实机
- 🛡️ 安全限位保护和摩擦补偿
- ⚡ 支持阻塞/非阻塞模式，实现并行运动

数据流架构：



SDK 直接通过 Zenoh 发布 JSON 数据，Bridge 节点负责转换为 ROS2 消息（如果使用 RViz 或实机）。

2)安装 SDK

代码块

```

1  git clone https://github.com/BlueWorm-EAI-Tech/bw_std_SDK.git standard_SDK
2  cd standard-SDK
3  python3 -m pip install -e .

```

Python 3.10 版本支持。

3)操作教程

代码块

```

1  from mantis import Mantis
2
3  with Mantis(ip="192.168.50.100") as robot:
4      print(robot.robot_sn, robot.robot_ip, robot.robot_version)

```

```

5
6     # 单关节命令只更新左臂的 Degree1，不会覆盖右臂目标。
7     robot.left_arm.set_shoulder_pitch(-0.2)
8
9     # 顺序固定为 Degree1 ~ Degree7。
10    robot.right_arm.set_joints([0.0, -0.2, 0.0, 0.3, 0.0, 0.0, 0.0])
11
12    robot.left_gripper.open()
13    robot.head.set_pose(pitch=0.0, yaw=0.2, roll=0.0)
14    robot.chassis.forward(0.05, speed=0.05)

```

方法	说明
<code>ip="192.168.50.100"</code>	指定 IP（推荐）
无参数	自动局域网发现
<code>ip + port</code>	指定 IP 和端口
<code>connect(verify=False)</code>	调试跳过验证

4)主类与模块

1 Mantis (主类)

属性	类型	说明
<code>left_arm / right_arm</code>	Arm	手臂控制器
<code>left_gripper / right_gripper</code>	Gripper	夹爪控制器
<code>head</code>	Head	头部控制器
<code>waist</code>	Waist	腰部控制器
<code>chassis</code>	Chassis	底盘控制器
<code>is_sim_mode</code>	bool	是否仿真模式

- `connect(timeout=5.0, verify=True)` 连接机器人
- `disconnect()` 断开连接
- `on_feedback(callback)` 注册关节反馈回调
- `home(block=True)` 所有关节归零
- `stop()` 停止所有运动
- `wait()` 等待所有部件运动完成
- `is_any_moving` 检查是否有部件在运动

2 Arm (手臂)

- 7 自由度关节，每个关节有限位保护
- 常用方法：
 - `set_joint(index, angle, block=True)` 设置单关节
 - `set_joints([angles], block=True)` 设置全部关节
 - `home()` 回零
 - `wait()` 等待完成
 - `set_speed(speed)` 设置速度

3 Gripper (夹爪)

- 位置范围：0.0（闭合） ~ 1.0（张开）
- 方法：
 - `open()`, `close()`, `half_open()`
 - `set_position(pos, block=True)`
 - `wait()`, `is_moving`

4 Head (头部)

- 限位：pitch -0.7~0.2, yaw -1.57~1.57 rad
- 方法：
 - `set_pose(pitch, yaw, block=True)`
 - `look_left/right/up/down()`
 - `center()`, `wait()`, `is_moving`

5 Body (躯干)

- prismatic 直线关节，高度范围 -0.62~0.24 m
- 方法：
 - `set_height(height, block=True)`
 - `up(delta)`, `down(delta)`
 - `home()`, `wait()`, `is_moving`

6 Chassis (底盘)

- 所有运动命令基于距离/角度，运动完成自动停止
- 方法：
 - `forward(distance, speed=1.5, block=True)`

- `backward()`, `strafe_left/right()`, `turn_left/right()`
 - `move(x, y, angle, block=True)` 组合运动
 - `set_friction(linear, angular)` 摩擦补偿
 - `stop()`, `wait()`, `is_moving`
-

5)控制示例

1 手臂控制

代码块

```
1 robot.left_arm.set_joints([0.5,0.2,0.1,0.8,0.0,0.3,0.0])
2 robot.left_arm.home()
3 robot.right_arm.home()
```

2 夹爪控制

代码块

```
1 robot.left_gripper.open()
2 robot.right_gripper.close()
3 robot.left_gripper.set_position(0.7)
```

3 头部控制

代码块

```
1 robot.head.look_left()
2 robot.head.look_right()
3 robot.head.set_pose(pitch=0.2, yaw=-0.3)
```

4 底盘控制

代码块

```
1 robot.chassis.forward(0.5)
2 robot.chassis.turn_left(90)
3 robot.chassis.move(x=0.5, y=0.2, angle=45)
4 robot.chassis.set_friction(linear=1.5, angular=2.0)
```

5 并行运动（非阻塞）

```

代码块
1 robot.left_arm.set_shoulder_pitch(-0.5, block=False)
2 robot.right_arm.set_shoulder_pitch(-0.5, block=False)
3 robot.head.look_left(block=False)
4 robot.wait() # 等待所有完成

```

6 关节反馈

代码块

```

1 robot.on_feedback(lambda names, pos: print(names, pos))
2 time.sleep(10) # 接收反馈

```

7 综合演示

代码块

```

1 robot.head.look_left()
2 robot.head.center()
3 robot.left_arm.set_shoulder_pitch(-0.5, block=False)
4 robot.right_arm.set_shoulder_pitch(-0.5, block=False)
5 robot.wait()
6 robot.left_gripper.open(block=False)
7 robot.right_gripper.open(block=False)
8 robot.wait()
9 robot.chassis.forward(0.3)
10 robot.chassis.turn_left(90)
11 robot.home()

```

Mantis Teleop (ROS2) 开发教程

提供 Standard 机器人的运动学解析和整机控制能力

代码块

```

1 src/
2 |— bw_std_description/ # URDF、mesh、mimic、限位与 ros2_control 契约
3 |— bw_std_control/    # V3 串口、SystemInterface、底盘与头部控制器
4 |— bw_teleop/         # 单节点 PICO UDP 接入与遥操作编排
5 |— bw_kinematics/     # 双臂 Pinocchio FK 与 CasADi SQP 联合 IK
6 |— bw_std_bringup/    # 控制器配置、真机和 mock 启动入口

```

遥操作数据流向如下

代码块

```
1 运行时数据流
2 PICO UDP
3    -> bw_teleop
4        |-- /target_left_pose, /target_right_pose -> bw_kinematics
5        |                                     -> /dual_arm_controller/joint_trajectory
6        |-- /gripper_controller/commands
7        |-- /lift_controller/commands
8        |-- /base_controller/reference
9        |-- /Teleop/head_pose
10       |-- /teleop/control_active -> base_controller -> safety/power
11       '--- /kinematics/reset_arms -> 关节空间复位
12 controller_manager -> StandardSystemHardware -> V3(0x01/0x21)
```

- 环境与构建

支持 Ubuntu 22.04 / ROS 2 Humble, 兼容 Ubuntu 24.04 / Jazzy:

代码块

```
1 # 执行 sudo、修改 Zsh 配置并构建，执行前必须输入 `APPLY` 二次确认。
2 ./scripts/local/ros_env_setup.sh --apply
3 ./scripts/local/ros_env_setup.sh --check # 只读检查环境。
```

代码块

```
1 echo $ROS_DISTRO # 确认 ROS2 版本source /opt/ros/$ROS_DISTRO/setup.zsh
2 rosdep install --from-paths src --ignore-src -r -y --skip-keys casadi
3 colcon --log-base log/$ROS_DISTRO build \
4   --build-base build/$ROS_DISTRO \
5   --install-base install/$ROS_DISTRO \
6   --symlink-install
7 source install/$ROS_DISTRO/setup.zsh
8 colcon test --build-base build/$ROS_DISTRO
9 colcon test-result --test-result-base build/$ROS_DISTRO --verbose
```

CasADi 没有上游 rosdep 规则, 由 `ros_env_setup.sh` 在统一前缀构建安装并校验运行时链接, 不从系统或用户目录混用 ABI.

- 启动与安全

```
代码块 真机：默认不上电、不启动遥操作、运动控制器 inactive.
2  ros2 launch bw_std_bringup standard.launch.py
3  # Mock：自动激活双臂、夹爪、升降、底盘和头部，并启动遥操作/运动学。
4  ros2 launch bw_std_bringup standard_mock.launch.py
```

六、注意事项

- 角度单位：弧度（rad），底盘旋转角度为度（°）
- 速度单位：线速度 m/s，角速度 rad/s
- 阻塞模式默认 block=True，非阻塞使用 block=False
- 摩擦补偿：地面摩擦大时增加运动时间
- 底盘运动自动停止，无需手动 stop()
- 连接超时可通过 `connect(timeout=10)` 调整
- 00000调试跳过验证：`connect(verify=False)`

《Mantis Pro人形机器人开发文档》版本信息

- 当前版本：V202609010
- 更新日期：2026 年 09 月 10 日
- 生效日期：2026 年 09 月 10 日

修订记录

版本号	修订日期	修订内容
V20260910	2026.09.10	修订开发文档全内容，覆盖产品概述、接口、

(注：部分内容可能由 AI 生成)